**REGISTRO DE BANCO DE DADOS - SEMANA 27**

**Obs.:** Atividade realizada pelo Codespace - GitHub

**Relatório/ E-mail:**

Olá,

Segue o relatório da atividade prática **"Explorando e aplicando estratégias de otimização de consultas no MongoDB"**, com o diagnóstico inicial, as estratégias utilizadas e os resultados obtidos.

**1. Diagnóstico inicial**

Foi criada a base de dados store, com a coleção products, contendo 10.000 documentos simulados com informações de produtos, categorias, preços, estoque, data de criação e avaliações.

Inicialmente, as consultas foram analisadas utilizando o método explain("executionStats") e o profiler do MongoDB. Os resultados indicaram que as consultas realizavam varreduras completas da coleção, identificadas pelo estágio COLLSCAN.

Na consulta que buscava produtos da categoria "eletrônicos", o MongoDB examinava os 10.000 documentos para retornar 2.020 resultados. Na consulta que filtrava produtos com preços entre 100 e 500 e os ordenava pela data de criação, também eram examinados 10.000 documentos, com 2.058 resultados retornados.

**2. Estratégias de otimização**

Para melhorar o desempenho das consultas, foram criados dois índices:

- category_1, utilizando o campo category;
- price_1_created_at_-1, utilizando os campos price e created_at.

Após a criação dos índices, as consultas foram executadas novamente com explain("executionStats") para verificar os planos de execução e a quantidade de documentos examinados.

**3. Comparação dos resultados**

**Consulta 1 — Produtos da categoria "eletrônicos"**

- Antes da otimização: 10.000 documentos examinados, com o plano COLLSCAN.
- Depois da otimização: 2.020 documentos examinados, com o índice category_1 e o estágio IXSCAN.
- Tempo registrado após a otimização: 3 ms.

Houve uma redução de 79,8% na quantidade de documentos examinados, demonstrando que o índice melhorou a eficiência dessa consulta.

**Consulta 2 — Produtos com preços entre 100 e 500, ordenados por data**

- Antes da otimização: 10.000 documentos examinados, com o plano COLLSCAN.
- Depois da otimização: 2.058 documentos examinados, utilizando o índice price_1_created_at_-1.
- Tempo registrado após a otimização: 8 ms.

A quantidade de documentos examinados diminuiu 79,42%. Entretanto, o plano de execução continuou apresentando o estágio SORT, indicando que a ordenação ainda era realizada separadamente pelo MongoDB.

**4. Análise do pipeline de agregação**

Foi implementado o pipeline com os estágios $unwind e $group, utilizando o operador $avg para calcular a média das avaliações por categoria.

O resultado apresentou as seguintes médias:

- Livros: 3,04
- Moda: 3,03
- Eletrônicos: 2,99
- Esportes: 3,03
- Casa: 3,00

Na análise com explain("executionStats"), a agregação examinou os 10.000 documentos da coleção, expandiu os arrays para processar 24.887 avaliações e produziu cinco resultados, um para cada categoria. O tempo informado para a etapa inicial da execução foi de 16 ms, sem uso de disco temporário.

O estágio COLLSCAN permaneceu presente porque o pipeline precisa percorrer os documentos e suas avaliações para calcular as médias.

**5. Conclusão**

A atividade demonstrou a importância dos índices para otimizar consultas no MongoDB. A utilização dos índices reduziu significativamente a quantidade de documentos examinados nas duas consultas testadas.

Também foi possível compreender como utilizar o profiler e o método explain("executionStats") para identificar gargalos e avaliar os planos de execução. A análise do pipeline mostrou que nem toda operação de agregação é necessariamente otimizada por índices, especialmente quando é necessário processar toda a coleção.

Como boas práticas, recomenda-se analisar as consultas com frequência, criar índices adequados aos filtros e às ordenações mais utilizados e acompanhar o desempenho após cada alteração.

Atenciosamente,

**Letícia Sá e Julia Alves, 3º B DS**

## Evidências/ Repositório - GitHub:




